

Node.js with MySQL

Complete Database Notes

Simple CRUD • Relational CRUD • Department & Employee System

Covers: MySQL Setup • mysql2 Package • Connection Pooling • CRUD Operations • JOINS • Relational Data • Express REST API • Best Practices

by

Sir Muhammad Farhan

1. Introduction — Node.js with MySQL

Why MySQL with Node.js?

MySQL is the world's most popular open-source relational database. When combined with Node.js, it is excellent for applications that need structured data, complex queries, and relationships between tables.

Feature	Details
Database Type	Relational (SQL)
Language	SQL (Structured Query Language)
Data Storage	Tables with rows and columns
Relationships	Foreign Keys, JOINS
Node.js Package	mysql2 (recommended)
Use Cases	E-commerce, ERP, HR systems, Blogs

MySQL vs MongoDB (Quick Comparison)

MySQL	MongoDB
Structured schema	Flexible / schema-less
SQL queries	NoSQL / JSON queries
Strong relationships (JOINS)	Embedded documents
ACID transactions	BASE model
Best for: financial, HR apps	Best for: real-time, flexible data

2. Setting Up MySQL with Node.js

Step 1 — Install MySQL Server

- Windows: Download installer from mysql.com
- Ubuntu/Debian: `sudo apt install mysql-server`
- Mac: `brew install mysql`

Step 2 — Install mysql2 Package

Always use `mysql2` instead of the older `mysql` package. It supports Promises and `async/await` natively.

```
npm init -y
npm install mysql2
npm install express dotenv # for REST API later
```

Step 3 — Create Database and User

```
-- Run in MySQL terminal (mysql -u root -p)

CREATE DATABASE company_db;

CREATE USER 'node_user'@'localhost' IDENTIFIED BY 'password123';
GRANT ALL PRIVILEGES ON company_db.* TO 'node_user'@'localhost';
FLUSH PRIVILEGES;

USE company_db;
```

Step 4 — Connect Node.js to MySQL

Simple Connection (Single)

```
// db.js
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'node_user',
  password: 'password123',
  database: 'company_db'
});

connection.connect((err) => {
  if (err) {
    console.error('Connection failed:', err.message);
    return;
  }
  console.log('MySQL Connected!');
```

```
});  
  
module.exports = connection;
```

Connection Pool (Recommended for Production)

A connection pool maintains multiple reusable connections, improving performance for high-traffic applications.

```
// db.js  
const mysql = require('mysql2/promise');  
  
const pool = mysql.createPool({  
  host: process.env.DB_HOST || 'localhost',  
  user: process.env.DB_USER || 'node_user',  
  password: process.env.DB_PASS || 'password123',  
  database: process.env.DB_NAME || 'company_db',  
  waitForConnections: true,  
  connectionLimit: 10,    // max 10 simultaneous connections  
  queueLimit: 0           // unlimited queued requests  
});  
  
module.exports = pool;
```

Always use Connection Pool in real applications. It is more efficient than creating a new connection for every request.

3. MySQL Data Types & SQL Basics

Common MySQL Data Types

Data Type	Description & Example
INT	Integer numbers. id INT AUTO_INCREMENT
VARCHAR(n)	Variable string up to n chars. name VARCHAR(100)
TEXT	Long text. description TEXT
DECIMAL(p,s)	Exact decimal. salary DECIMAL(10,2)
BOOLEAN / TINYINT(1)	True/False. is_active BOOLEAN DEFAULT TRUE
DATE	Date only. hire_date DATE
DATETIME	Date and time. created_at DATETIME
TIMESTAMP	Auto-updated timestamp. updated_at TIMESTAMP
ENUM	Predefined values. status ENUM('active','inactive')

Essential SQL Commands

```
-- CREATE table
CREATE TABLE users (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  email   VARCHAR(150) NOT NULL UNIQUE,
  age     INT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- INSERT
INSERT INTO users (name, email, age) VALUES ('Alice', 'alice@mail.com', 25);

-- SELECT
SELECT * FROM users;
SELECT id, name FROM users WHERE age > 20 ORDER BY name ASC;

-- UPDATE
UPDATE users SET age = 26 WHERE id = 1;

-- DELETE
DELETE FROM users WHERE id = 1;

-- DROP table
DROP TABLE IF EXISTS users;
```

4. Simple CRUD — Users Table

Create the Table

```
-- Run in MySQL
USE company_db;

CREATE TABLE users (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL UNIQUE,
  age         INT,
  is_active   BOOLEAN DEFAULT TRUE,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

Project Structure

```
simple-crud/
├─ db.js          # Database connection pool
├─ app.js         # Express app & routes
├─ .env           # Environment variables
└─ package.json
```

db.js — Connection Pool

```
// db.js
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host:      process.env.DB_HOST || 'localhost',
  user:      process.env.DB_USER || 'node_user',
  password:  process.env.DB_PASS || 'password123',
  database:  process.env.DB_NAME || 'company_db',
  connectionLimit: 10
});

module.exports = pool;
```

CREATE — Insert a User

```
// app.js
const express = require('express');
const pool    = require('./db');
const app     = express();
app.use(express.json());
```

```
// POST /api/users - Create new user
app.post('/api/users', async (req, res) => {
  try {
    const { name, email, age } = req.body;

    // Validate required fields
    if (!name || !email) {
      return res.status(400).json({ error: 'name and email are required' });
    }

    const sql = 'INSERT INTO users (name, email, age) VALUES (?, ?, ?)';
    const [result] = await pool.execute(sql, [name, email, age]);

    res.status(201).json({
      message: 'User created successfully',
      userId: result.insertId
    });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({ error: 'Email already exists' });
    }
    res.status(500).json({ error: err.message });
  }
});
```

Always use parameterized queries (? placeholders). Never concatenate user input directly into SQL strings — that causes SQL Injection attacks.

READ — Get All Users & Get by ID

```
// GET /api/users - Get all users
app.get('/api/users', async (req, res) => {
  try {
    const [rows] = await pool.execute('SELECT * FROM users ORDER BY created_at DESC');
    res.json({ count: rows.length, users: rows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET /api/users/:id - Get single user
app.get('/api/users/:id', async (req, res) => {
  try {
    const [rows] = await pool.execute(
      'SELECT * FROM users WHERE id = ?',
      [req.params.id]
    );

    if (rows.length === 0) {
      return res.status(404).json({ error: 'User not found' });
    }
  }
});
```

```

    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET /api/users?search=alice - Search users
app.get('/api/users/search', async (req, res) => {
  try {
    const search = '%' + (req.query.q || '') + '%';
    const [rows] = await pool.execute(
      'SELECT * FROM users WHERE name LIKE ? OR email LIKE ?',
      [search, search]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

UPDATE — Update a User

```

// PUT /api/users/:id - Full update
app.put('/api/users/:id', async (req, res) => {
  try {
    const { name, email, age, is_active } = req.body;
    const { id } = req.params;

    const sql = `
      UPDATE users
      SET name = ?, email = ?, age = ?, is_active = ?
      WHERE id = ?
    `;
    const [result] = await pool.execute(sql, [name, email, age, is_active, id]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'User not found' });
    }

    res.json({ message: 'User updated successfully' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// PATCH /api/users/:id - Partial update
app.patch('/api/users/:id', async (req, res) => {
  try {
    const { id } = req.params;
    const fields = req.body; // only provided fields

```



```

// Build dynamic SET clause
const columns = Object.keys(fields).map(k => `${k} = ?`).join(', ');
const values  = [...Object.values(fields), id];

const [result] = await pool.execute(
  `UPDATE users SET ${columns} WHERE id = ?`, values
);

if (result.affectedRows === 0) {
  return res.status(404).json({ error: 'User not found' });
}

res.json({ message: 'User partially updated' });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

```

DELETE — Remove a User

```

// DELETE /api/users/:id
app.delete('/api/users/:id', async (req, res) => {
  try {
    const [result] = await pool.execute(
      'DELETE FROM users WHERE id = ?',
      [req.params.id]
    );

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'User not found' });
    }

    res.json({ message: 'User deleted successfully' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

app.listen(3000, () => console.log('Server running on port 3000'));

```

Testing with Postman / curl

Operation	Request
Create User	POST http://localhost:3000/api/users
Get All	GET http://localhost:3000/api/users
Get One	GET http://localhost:3000/api/users/1

Update	PUT http://localhost:3000/api/users/1
Part. Update	PATCH http://localhost:3000/api/users/1
Delete	DELETE http://localhost:3000/api/users/1

5. Relational CRUD — Department & Employee

Understanding the Relationship

A Department can have many Employees. An Employee belongs to one Department. This is a One-to-Many (1:N) relationship. The employees table holds a foreign key department_id that references the departments table.

Database Schema (ERD Description)

departments	
id	INT PK AUTO_INCREMENT
name	VARCHAR(100) NOT NULL UNIQUE
location	VARCHAR(100)
budget	DECIMAL(15,2)
created_at	TIMESTAMP
employees	
id	INT PK AUTO_INCREMENT
first_name	VARCHAR(100) NOT NULL
last_name	VARCHAR(100) NOT NULL
email	VARCHAR(150) NOT NULL UNIQUE
phone	VARCHAR(20)
salary	DECIMAL(10,2)
hire_date	DATE
department_id	INT FK → departments(id)
created_at	TIMESTAMP
Relationship: departments.id ← employees.department_id (ONE to MANY)	

Create Tables in MySQL

USE company_db;
-- 1. Departments table (parent)
CREATE TABLE departments (
id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL UNIQUE,
location VARCHAR(100),
budget DECIMAL(15, 2) DEFAULT 0.00,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- 2. Employees table (child — has foreign key)
CREATE TABLE employees (
id INT AUTO_INCREMENT PRIMARY KEY,
first_name VARCHAR(100) NOT NULL,
last_name VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL UNIQUE,

```

phone          VARCHAR(20),
salary         DECIMAL(10, 2) DEFAULT 0.00,
hire_date      DATE,
department_id  INT,
created_at     TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

-- Foreign Key Constraint
CONSTRAINT fk_department
    FOREIGN KEY (department_id)
    REFERENCES departments(id)
    ON DELETE SET NULL      -- if dept deleted, employee stays (dept = null)
    ON UPDATE CASCADE      -- if dept id changes, employee updates automatically
);

-- Seed sample data
INSERT INTO departments (name, location, budget) VALUES
    ('Engineering', 'Karachi', 5000000.00),
    ('Marketing', 'Lahore', 2000000.00),
    ('HR', 'Islamabad', 1500000.00);

INSERT INTO employees (first_name, last_name, email, phone, salary, hire_date,
    department_id) VALUES
    ('Ali', 'Khan', 'ali@company.com', '0300-1234567', 85000, '2022-01-15',
    1),
    ('Sara', 'Ahmed', 'sara@company.com', '0301-2345678', 75000, '2022-03-10',
    1),
    ('Usman', 'Malik', 'usman@company.com', '0302-3456789', 65000, '2023-06-01',
    2),
    ('Fatima', 'Sheikh', 'fatima@company.com', '0303-4567890', 70000, '2023-08-20',
    3);

```

6. Department CRUD — Full Implementation

Project Structure

```
company-api/  
├─ db.js  
├─ routes/  
│   ├─ departments.js  
│   └─ employees.js  
├─ app.js  
├─ .env  
└─ package.json
```

routes/departments.js

```
const express = require('express');  
const router  = express.Router();  
const pool    = require('../db');  
  
// — CREATE Department —  
// POST /api/departments  
router.post('/', async (req, res) => {  
  try {  
    const { name, location, budget } = req.body;  
  
    if (!name) return res.status(400).json({ error: 'Department name is required' });  
  
    const [result] = await pool.execute(  
      'INSERT INTO departments (name, location, budget) VALUES (?, ?, ?)',  
      [name, location || null, budget || 0]  
    );  
  
    res.status(201).json({  
      message: 'Department created',  
      departmentId: result.insertId  
    });  
  } catch (err) {  
    if (err.code === 'ER_DUP_ENTRY') {  
      return res.status(409).json({ error: 'Department name already exists' });  
    }  
    res.status(500).json({ error: err.message });  
  }  
});  
  
// — READ All Departments (with employee count) —  
// GET /api/departments  
router.get('/', async (req, res) => {  
  try {  
    const sql = `  
      SELECT
```

```

        d.id,
        d.name,
        d.location,
        d.budget,
        COUNT(e.id) AS employee_count,
        d.created_at
    FROM departments d
    LEFT JOIN employees e ON e.department_id = d.id
    GROUP BY d.id
    ORDER BY d.name ASC
`;

    const [rows] = await pool.execute(sql);
    res.json({ count: rows.length, departments: rows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — READ Single Department —————
// GET /api/departments/:id
router.get('/:id', async (req, res) => {
  try {
    const [rows] = await pool.execute(
      'SELECT * FROM departments WHERE id = ?',
      [req.params.id]
    );
    if (rows.length === 0) {
      return res.status(404).json({ error: 'Department not found' });
    }
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// — UPDATE Department —————
// PUT /api/departments/:id
router.put('/:id', async (req, res) => {
  try {
    const { name, location, budget } = req.body;
    const [result] = await pool.execute(
      'UPDATE departments SET name = ?, location = ?, budget = ? WHERE id = ?',
      [name, location, budget, req.params.id]
    );
    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Department not found' });
    }
    res.json({ message: 'Department updated' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```
// --- DELETE Department ---
// DELETE /api/departments/:id
router.delete('/:id', async (req, res) => {
  try {
    // Check if department has employees
    const [empRows] = await pool.execute(
      'SELECT COUNT(*) AS total FROM employees WHERE department_id = ?',
      [req.params.id]
    );

    if (empRows[0].total > 0) {
      return res.status(400).json({
        error: `Cannot delete. ${empRows[0].total} employee(s) belong to this
department.`
      });
    }

    const [result] = await pool.execute(
      'DELETE FROM departments WHERE id = ?',
      [req.params.id]
    );
    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Department not found' });
    }
    res.json({ message: 'Department deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

7. Employee CRUD — Full Implementation

routes/employees.js

```
const express = require('express');
const router = express.Router();
const pool = require('../db');

// — CREATE Employee —
// POST /api/employees
router.post('/', async (req, res) => {
  try {
    const { first_name, last_name, email, phone, salary, hire_date, department_id } = req.body;

    if (!first_name || !last_name || !email) {
      return res.status(400).json({ error: 'first_name, last_name and email are required' });
    }

    // Validate department exists
    if (department_id) {
      const [dept] = await pool.execute(
        'SELECT id FROM departments WHERE id = ?', [department_id]
      );
      if (dept.length === 0) {
        return res.status(404).json({ error: 'Department not found' });
      }
    }

    const sql = `
      INSERT INTO employees (first_name, last_name, email, phone, salary, hire_date, department_id)
      VALUES (?, ?, ?, ?, ?, ?, ?)`;
    const [result] = await pool.execute(sql, [
      first_name, last_name, email,
      phone || null,
      salary || 0,
      hire_date || null,
      department_id || null
    ]);

    res.status(201).json({
      message: 'Employee created',
      employeeId: result.insertId
    });
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') {
      return res.status(409).json({ error: 'Email already exists' });
    }
    res.status(500).json({ error: err.message });
  }
}
```



```
});
```

READ Employees — with JOIN (Department Info)

```
// GET /api/employees - All employees with department name
router.get('/', async (req, res) => {
  try {
    const sql = `
      SELECT
        e.id,
        e.first_name,
        e.last_name,
        e.email,
        e.phone,
        e.salary,
        e.hire_date,
        d.id AS department_id,
        d.name AS department_name,
        d.location AS department_location
      FROM employees e
      LEFT JOIN departments d ON e.department_id = d.id
      ORDER BY e.last_name ASC
    `;
    const [rows] = await pool.execute(sql);
    res.json({ count: rows.length, employees: rows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET /api/employees/:id - Single employee with department
router.get('/:id', async (req, res) => {
  try {
    const sql = `
      SELECT e.*, d.name AS department_name, d.location
      FROM employees e
      LEFT JOIN departments d ON e.department_id = d.id
      WHERE e.id = ?
    `;
    const [rows] = await pool.execute(sql, [req.params.id]);
    if (rows.length === 0) {
      return res.status(404).json({ error: 'Employee not found' });
    }
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// GET /api/employees/department/:deptId - All employees in a dept
router.get('/department/:deptId', async (req, res) => {
  try {
```

```

const sql = `
  SELECT e.*, d.name AS department_name
  FROM employees e
  INNER JOIN departments d ON e.department_id = d.id
  WHERE e.department_id = ?
  ORDER BY e.salary DESC`,
const [rows] = await pool.execute(sql, [req.params.deptId]);
res.json({ count: rows.length, employees: rows });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

```

UPDATE Employee

```

// PUT /api/employees/:id
router.put('/:id', async (req, res) => {
  try {
    const { first_name, last_name, email, phone, salary, hire_date, department_id }
    = req.body;

    // Validate department exists before updating
    if (department_id) {
      const [dept] = await pool.execute('SELECT id FROM departments WHERE id = ?',
[department_id]);
      if (dept.length === 0) return res.status(404).json({ error: 'Department not
found' });
    }

    const sql = `
      UPDATE employees
      SET first_name = ?, last_name = ?, email = ?,
        phone = ?, salary = ?, hire_date = ?, department_id = ?
      WHERE id = ?
    `;
    const [result] = await pool.execute(sql, [
      first_name, last_name, email,
      phone, salary, hire_date, department_id, req.params.id
    ]);

    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Employee not found' });
    }
    res.json({ message: 'Employee updated' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// PATCH /api/employees/:id - Transfer to different department
router.patch('/:id/transfer', async (req, res) => {
  try {

```

```

const { department_id } = req.body;

// Validate new department
const [dept] = await pool.execute('SELECT id, name FROM departments WHERE id = ?', [department_id]);
if (dept.length === 0) return res.status(404).json({ error: 'Department not found' });

await pool.execute('UPDATE employees SET department_id = ? WHERE id = ?', [department_id, req.params.id]);

res.json({ message: `Employee transferred to ${dept[0].name}` });
} catch (err) {
  res.status(500).json({ error: err.message });
}
});

```

DELETE Employee

```

// DELETE /api/employees/:id
router.delete('/:id', async (req, res) => {
  try {
    const [result] = await pool.execute(
      'DELETE FROM employees WHERE id = ?',
      [req.params.id]
    );
    if (result.affectedRows === 0) {
      return res.status(404).json({ error: 'Employee not found' });
    }
    res.json({ message: 'Employee deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;

```

8. Main app.js — Putting It All Together

```
// app.js
const express = require('express');
const app = express();
require('dotenv').config();

// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

// Routes
const departmentRoutes = require('./routes/departments');
const employeeRoutes = require('./routes/employees');

app.use('/api/departments', departmentRoutes);
app.use('/api/employees', employeeRoutes);

// Health check
app.get('/', (req, res) => {
  res.json({ message: 'Company API is running', version: '1.0.0' });
});

// 404 handler
app.use((req, res) => {
  res.status(404).json({ error: 'Route not found' });
});

// Global error handler
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Internal Server Error' });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

.env File

```
PORT=3000
DB_HOST=localhost
DB_USER=node_user
DB_PASS=password123
DB_NAME=company_db
```

9. Advanced Queries & Reporting

JOIN Types Explained

JOIN Type	Returns
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All rows from left + matching from right (NULL if no match)
RIGHT JOIN	All rows from right + matching from left (NULL if no match)
CROSS JOIN	Every combination of rows from both tables

Report 1 — Department Summary

```
// GET /api/reports/departments
router.get('/departments', async (req, res) => {
  const sql = `
    SELECT
      d.name           AS department,
      d.location,
      d.budget,
      COUNT(e.id)      AS total_employees,
      AVG(e.salary)    AS avg_salary,
      MAX(e.salary)    AS highest_salary,
      MIN(e.salary)    AS lowest_salary,
      SUM(e.salary)    AS total_payroll
    FROM departments d
    LEFT JOIN employees e ON e.department_id = d.id
    GROUP BY d.id
    ORDER BY total_employees DESC
  `;
  const [rows] = await pool.execute(sql);
  res.json(rows);
});
```

Report 2 — Employees with Filters

```
// GET /api/employees?dept=Engineering&minSalary=60000&maxSalary=100000
router.get('/', async (req, res) => {
  const { dept, minSalary, maxSalary } = req.query;
  let sql = `SELECT e.*, d.name AS dept_name FROM employees e`,
  let params = [];
  let where = [];

  sql += ' LEFT JOIN departments d ON e.department_id = d.id';

  if (dept) {
    where.push('d.name = ?');
    params.push(dept);
  }
```

```

}
if (minSalary) {
  where.push('e.salary >= ?');
  params.push(minSalary);
}
if (maxSalary) {
  where.push('e.salary <= ?');
  params.push(maxSalary);
}
if (where.length > 0) {
  sql += ' WHERE ' + where.join(' AND ');
}
sql += ' ORDER BY e.salary DESC';

const [rows] = await pool.execute(sql, params);
res.json({ count: rows.length, employees: rows });
});

```

Transactions — Atomic Operations

Transactions ensure that either all operations succeed or none of them do. Essential for financial or multi-step operations.

```

// Example: Transfer employee between departments + update budget
router.post('/transfer', async (req, res) => {
  const { employeeId, fromDeptId, toDeptId } = req.body;
  const connection = await pool.getConnection();

  try {
    await connection.beginTransaction();

    // Step 1: Move employee
    await connection.execute(
      'UPDATE employees SET department_id = ? WHERE id = ?',
      [toDeptId, employeeId]
    );

    // Step 2: Decrease headcount allowance in old dept
    await connection.execute(
      'UPDATE departments SET budget = budget - 5000 WHERE id = ?',
      [fromDeptId]
    );

    // Step 3: Increase in new dept
    await connection.execute(
      'UPDATE departments SET budget = budget + 5000 WHERE id = ?',
      [toDeptId]
    );

    await connection.commit(); // All steps succeeded
    res.json({ message: 'Transfer completed successfully' });
  } catch (error) {
    // Transaction failed, rollback
    await connection.rollback();
    res.status(500).json({ message: 'Transfer failed' });
  }
});

```

```
    } catch (err) {  
      await connection.rollback(); // Something failed - undo everything  
      res.status(500).json({ error: 'Transfer failed: ' + err.message });  
    } finally {  
      connection.release(); // Always release connection back to pool  
    }  
  });  
});
```

10. Complete API Endpoints Reference

Department Endpoints

Method + URL	Description
GET /api/departments	Get all departments (with employee count)
GET /api/departments/:id	Get single department by ID
POST /api/departments	Create new department
PUT /api/departments/:id	Update department (full)
DELETE /api/departments/:id	Delete department (if no employees)

Employee Endpoints

Method + URL	Description
GET /api/employees	Get all employees (with dept name)
GET /api/employees/:id	Get single employee with dept info
GET /api/employees/department/:deptId	Get all employees in a department
POST /api/employees	Create new employee
PUT /api/employees/:id	Update employee (full)
PATCH /api/employees/:id/transfer	Transfer employee to new dept
DELETE /api/employees/:id	Delete employee

Sample Request Bodies

POST /api/departments

```
{
  "name": "Finance",
  "location": "Karachi",
  "budget": 3000000
}
```

POST /api/employees

```
{
  "first_name": "Ahmed",
  "last_name": "Raza",
  "email": "ahmed@company.com",
  "phone": "0311-1234567",
  "salary": 80000,
  "hire_date": "2024-01-15",
}
```



```
"department_id": 1
}
```

Sample Responses

GET /api/employees (with JOIN)

```
{
  "count": 4,
  "employees": [
    {
      "id": 1,
      "first_name": "Ali",
      "last_name": "Khan",
      "email": "ali@company.com",
      "salary": "85000.00",
      "department_id": 1,
      "department_name": "Engineering",
      "department_location": "Karachi"
    }
  ]
}
```

11. Best Practices & Common Mistakes

Security

- Always use parameterized queries (?) — never string concatenate SQL
- Never store database credentials in code — use .env files
- Add .env and node_modules to .gitignore
- Validate all inputs before running queries
- Use HTTPS in production

Performance

- Use Connection Pool (mysql2/promise pool) — not single connections
- Add database indexes on frequently searched columns
- Select only needed columns: SELECT id, name — avoid SELECT *
- Use pagination (LIMIT and OFFSET) for large datasets
- Cache frequent read queries using Redis

Pagination Example

```
// GET /api/employees?page=1&limit=10
router.get('/', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const offset = (page - 1) * limit;

  const [rows] = await pool.execute(
    'SELECT * FROM employees LIMIT ? OFFSET ?',
    [limit, offset]
  );

  const [{ total }] = await pool.execute(
    'SELECT COUNT(*) AS total FROM employees'
  );

  res.json({
    page,
    limit,
    total,
    totalPages: Math.ceil(total / limit),
    employees: rows
  });
});
```

Common Mistakes to Avoid

Mistake	Correct Approach
---------	------------------

Using mysql (old package)	Use mysql2 — supports async/await
String concat in SQL	Use ? parameterized queries
No error handling	Always use try/catch in async routes
Not releasing connections	Use pool or always call release()
Storing passwords as plain text	Use bcrypt for password hashing
No input validation	Validate before running any query
SELECT * in production	Select only needed columns
No indexes on foreign keys	Add INDEX on department_id column

Add Index for Performance

```
-- Add index on foreign key for faster JOINS
CREATE INDEX idx_employee_dept ON employees(department_id);

-- Add index on frequently searched column
CREATE INDEX idx_employee_email ON employees(email);

-- Check indexes
SHOW INDEX FROM employees;
```

Quick Tip: Run EXPLAIN SELECT ... in MySQL to see how the query is executed and whether it is using indexes. This helps optimize slow queries.